Quantyx Pricer

Technical Reference

Pricing Models and Instrument Coverage

Platform	QuantyxPricer — QuantLib-based multi-model pricing workspace
Technology	Python · QuantLib · FastAPI · React.js · MySQL
Models	17 priceable instrument types across 12 model files
Dispatch	pricer.py — routes by "model" field in each instrument JSON
API	FastAPI REST — 20+ endpoints for pricing, data, and model management
Frontend	React.js — portfolio view, instrument editor, settings

Platform Overview

Quantyx Pricer is a fixed-income and structured-product pricing platform built on QuantLib. It supports a broad range of instrument types from investment-grade corporate bonds through complex equity-linked structured products. Each instrument is described by a JSON file (one per ISIN), and the pricing model is selected by the "model" field in that file. The root dispatcher pricer.py routes each instrument to the correct pricing engine, which reads the benchmark curves from curves/swap_curves.json and writes a PDF report to output/.

Architecture

The system has five layers: (1) instrument JSON files in assets/, (2) benchmark curve catalogue in curves/swap_curves.json, (3) model-specific Python pricers in models/, each exposing price_asset() and print_result(), (4) a FastAPI backend (api/main.py) providing REST endpoints for pricing, data management, and model configuration, and (5) a React frontend (website/) for portfolio management and instrument editing.

Instrument Taxonomy

Category	Model(s)	Instrument types
Rate-based bonds	`hullwhite, trinomialtree, montecarlo`	Fixed / floating / callable investment-grade bonds
Credit instruments	`cln, at1`	CLNs, Additional Tier 1 / CoCo perpetuals
Money-market	`discount_note`	Repos, commercial paper, T-bills
Leveraged / alt. credit	`pik, abs, clo`	PIK bonds, ABS, MBS, CLO tranches, leveraged loans
Structured equity-linked	`barrier_convertible, barrier_discount, simple_convertible, autocallable reverse convertible`	Monte Carlo equity payoffs
Inflation / index-linked	`index_linked, inflation_linked`	CMS channel notes, government linkers
Decomposition	`spire`	Collateral + swap structured notes

Common Conventions

All models share the following conventions. Dates accept DD-MM-YYYY or YYYY-MM-DD. Rates accept decimal (0.045) or percentage (4.5) — the helper auto-normalises values > 1. Spreads are always in basis points. The discount curve is selected automatically from curves/swap_curves.json by matching the instrument's currency to an OIS curve, or overridden via "discount_curve_name". Required fields highlighted in yellow in the tables below.

Yellow rows = required fields. White rows = optional fields with sensible defaults.

Data Providers

Quantyx Pricer integrates three external data providers, each responsible for a distinct data domain. Credentials are stored in a .env file at the project root and loaded at startup via python-dotenv. The provider module is located at api/provider.py and is consumed exclusively by the FastAPI layer — model pricers read only local files (curves/swap_curves.json, assets/) and never call external APIs directly.

Provider	Data domain	Instrument types	Auth
Refinitiv Datastream	CDS spread time series	CLN (hazard-rate calibration)	`DS_ID / DS_PWD`
CBonds	Bond metadata + trade prices / estimates	All fixed-income ISINs	`CBONDS_LOGIN / CBONDS_PASSWORD`
EODHD	Equity end-of-day prices	Equity underlyings (structured products)	`EODHD_API_KEY`

API routing logic: the /download_prices endpoint detects the asset type via the asset_type field. If "equity" → EODHD (using ticker or ISIN as code). Otherwise → CBonds. If the asset is not found in cache or DB, EODHD is tried first, then CBonds as a fallback. The /fetch_asset endpoint uses CBonds as the fallback when an ISIN is not found in the local MySQL database.

Refinitiv Datastream

`model: "api/provider.py"  ·  instrument_type: "fetch_from_ds"`

CDS spread time series — DatastreamPy SDK

Overview

Datastream (now part of LSEG / Refinitiv) is used to retrieve CDS spread time series for reference entities. These series feed the CLN model's hazard-rate calibration, where each CDS tenor spread is converted to a piecewise-constant hazard rate λ(t) = s(t) / (1 − R). Datastream is also available for benchmark curve pulls when the ECB swap-curve feed is insufficient for a given currency.

SDK and Authentication

The connection uses the DatastreamPy Python SDK (pip install DatastreamPy). A DataClient is instantiated per call with credentials sourced from environment variables DS_ID and DS_PWD (Datastream user ID and password). No persistent session is maintained; each function call creates and destroys its own client.

Data Request

Data is retrieved via ds.get_data(). The function signature accepts a Datastream ticker (e.g. a CDS mnemonic or bond code), a list of fields, a start date, an end date, and a frequency. The result is a pandas DataFrame which is immediately serialised to a list of records for API consumption.

```
df = ds.get_data(
    tickers = symbol,       # e.g. "DSUSD5Y=" for 5Y USD CDS
    fields  = ["P"],        # P = price / last value
    start   = "Y",          # 1 year ago (Datastream relative date)
    end     = "0D",         # today
    freq    = "D"           # daily
)
return json.loads(df.to_json(orient="records", date_format="iso"))
```

Date Syntax

Datastream date token	Meaning
`"Y"`	One year before today (Datastream relative)
`"0D"`	Today
`"6M"`	Six months before today
`"YYYY-MM-DD"`	Absolute date (ISO 8601)

Environment Variables

Field	Type	Description
DS_ID	string	Datastream user ID (e.g. "A12345")
DS_PWD	string	Datastream password

Output

Returns a list of JSON objects, one per trading day, each containing a Date field (ISO 8601 string) and a P field (the closing price or CDS spread value). This list is returned directly from the /fetch_asset_timeseries endpoint or consumed internally by the curve-update script.

```
[ {"Date": "2024-01-02T00:00:00.000Z", "P": 82.5},
```

{"Date": "2024-01-03T00:00:00.000Z", "P": 83.1}, ...]

CBonds

model: "api/provider.py" · instrument_type: "fetch_from_cbonds · fetch_prices_from_cbonds · fetch_estimates_from_cbonds"

Bond instrument metadata + trade prices + estimates

Overview

CBonds is used as the primary data source for all fixed-income instruments. It provides two distinct data types: (1) instrument metadata retrieved by ISIN from the emissions endpoint (issuer, coupon, maturity, rating, etc.) and (2) market prices and yield estimates retrieved from the tradings endpoint. All three CBonds functions use the same JSON-POST protocol over REST, differing only in the endpoint URL and sort field.

Base URLs

```
Emissions  (metadata):
https://ws.cbonds.info/services/json/get_emissions/?lang=eng
Tradings   (prices):
https://ws.cbonds.info/services/json/get_tradings_new/?lang=eng
Estimates  (bid/ask):
https://ws.cbonds.info/services/json/get_tradings_new/?lang=eng
```

Authentication

Authentication uses HTTP credentials passed as a JSON object in the request body — not as an HTTP Authorization header. Credentials come from environment variables CBONDS_LOGIN and CBONDS_PASSWORD.

Request Structure

Every CBonds call is an HTTP POST with Content-Type: application/json. The payload follows a common schema with four top-level keys:

```
{
    "auth":     { "login": "...", "password": "..." },
    "filters":  [ { "field": "isin_code",
                    "operator": "eq",    // or "in" for metadata
                    "value": "XS1234567890" } ],
    "quantity": { "limit": 10, "offset": 0 },
    "sorting":  [ { "field": "trade_date", "order": "desc" } ],
    "fields":   []   // empty = return all fields
}
```

Functions and Differences

Function	Endpoint	Filter operator	Sort field
fetch_from_cbonds	get_emissions	"in"	"id" asc
fetch_prices_from_cbonds	get_tradings_new	"eq"	"trade_date" desc
fetch_estimates_from_cbonds	get_tradings_new	"eq"	"date" desc

Response Parsing

The CBonds API returns a JSON object. The function inspects for either an "items" or "data" key (both are used depending on the endpoint version). Only the first result is returned — meaning the most recent record after the sort. The provider tag "cbonds" is injected before the dict is returned to the caller.

```
    data = response.json()
    results = data.get("items") or data.get("data") or []
    item = results[0]
    item["provider"] = "cbonds"
    return item
```

API Wiring

GET /fetch_asset: DB lookup first; if not found, calls fetch_from_cbonds(isin) and inserts the returned metadata into MySQL. GET /download_prices: for non-equity assets, calls fetch_from_cbonds(isin). GET /download_all_prices: iterates cached assets, calls fetch_from_cbonds for all non-equity entries.

Environment Variables

Field	Type	Description
CBONDS_LOGIN	string	CBonds account login (email or username)
CBONDS_PASSWORD	string	CBonds account password

Timeout and Error Handling

All CBonds requests use a 10-second timeout (30s for estimates). On RequestException (network error, timeout, non-2xx status) or JSONDecodeError the function logs the error and returns None. The API layer raises HTTP 404 if None is returned from both local DB and provider.

EODHD (End of Day Historical Data)

model: "api/provider.py" · instrument_type: "fetch_prices_from_eodhd"

Equity end-of-day prices for structured product underlyings

Overview

EODHD provides daily OHLCV price series for equities and stock indices. In Quantyx Pricer, EODHD feeds the underlying asset price histories used by structured product models (barrier_convertible, barrier_discount, simple_convertible, autocallable_reverse_convertible) for spot level verification and historical scenario analysis. EODHD is selected automatically when the instrument's asset_type field equals "equity".

Endpoint

```
Base URL:  https://eodhd.com/api/eod
Full URL:  https://eodhd.com/api/eod/{symbol}?api_token={EODHD_API_KEY}&fmt=json

Method: GET
Header: Accept: application/json
```

Authentication

Authentication uses a bearer-style API token passed as a URL query parameter (api_token). The token is read from the EODHD_API_KEY environment variable at module load time and embedded in every request URL. No additional headers are required.

Symbol Format

The symbol passed to fetch_prices_from_eodhd is resolved from the asset record in the following priority order: ticker field → isin_code field → instrument_id. EODHD tickers follow the pattern TICKER.EXCHANGE (e.g. AAPL.US, VOW3.XETRA, NESN.SW). ISINs can also be passed directly for many instruments.

Response Format

The fmt=json parameter causes EODHD to return a JSON array. Each element represents one trading day with the following fields:

```
[
  {
    "date":            "2024-01-02",
    "open":            185.24,
    "high":            186.10,
    "low":             184.92,
    "close":           185.85,
    "adjusted_close":  184.71,
    "volume":          54321000
  }, ...
]
```

The provider wraps the list in a dict and injects provider and instrument_id keys before returning to the API caller:

```
return {
    "provider":      "eodhd",
    "instrument_id": symbol,
    "data":          [ ...daily OHLCV records... ]
```

```
}
```

API Wiring

GET /download_prices: if _is_equity_asset(asset) is True, calls fetch_prices_from_eodhd(ticker). The _is_equity_asset() helper checks asset["asset_type"] == "equity". If the asset is not found in cache or DB, EODHD is tried first (as it covers tickers like AAPL that do not have ISINs in CBonds), then CBonds as a fallback.

Environment Variables

Field	Type	Description
`EODHD_API_KEY`	string	EODHD API token (from your eodhd.com account)

Timeout and Error Handling

Requests timeout after 10 seconds. On RequestException, JSONDecodeError, or any other unexpected exception the function logs the error with [Provider] prefix and returns None. If both EODHD and CBonds return None for an asset, the API returns HTTP 404.

All provider credentials must be present in a .env file at the project root. The file is never committed to version control. Required keys: DS_ID, DS_PWD (Datastream) · CBONDS_LOGIN, CBONDS_PASSWORD (CBonds) · EODHD_API_KEY (EODHD).

MySQL Database

model: "api/db.py" · instrument_type: "mysql.connector · utf8mb4"

Persistence layer — three tables, JSON-blob storage, stored procedures

Overview

Quantyx Pricer uses MySQL 8+ as its persistence layer. All business data (instrument definitions, pricing results, model schemas) is stored as opaque JSON blobs, keeping the relational schema minimal and schema-free. The database module is api/db.py. It uses the mysql.connector Python library with a new connection opened and closed per operation — there is no connection pool. All connections use the utf8mb4 charset (full Unicode, including multi-byte characters in issuer names).

Connection

Connection parameters are sourced exclusively from environment variables. The _required_env() helper raises ValueError immediately at startup if any mandatory variable is absent, so misconfigured deployments fail fast rather than silently.

```
mysql.connector.connect(
    host     = DB_SERVER,       # hostname or IP
    user     = DB_USER,
    password = DB_PASS,
    database = DB_DATABASE,
    port     = int(DB_PORT or 3306),
    charset  = "utf8mb4",
    use_unicode = True,
)
```

Field	Type	Description
DB_SERVER	string	MySQL hostname or IP address
DB_USER	string	MySQL user with EXECUTE privilege on stored procedures
DB_PASS	string	MySQL password
DB_DATABASE	string	Target database / schema name
DB_PORT	int	MySQL port (default 3306)

Table: assets

Stores one row per instrument. The code column holds the ISIN (or internal ID) used as the lookup key. The json column holds the full instrument JSON as returned by the asset editor or imported from CBonds.

```
CREATE TABLE assets (
    my_row_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    code       VARCHAR(64)  NOT NULL,    -- ISIN or internal ID
    json       JSON         NOT NULL     -- full instrument definition
);
```

Key operations:

```
SELECT my_row_id, code, json FROM assets;              -- select_assets()
SELECT code, json FROM assets WHERE code=%s LIMIT 1;   -- select_asset(code)
CALL insert_asset(p_json JSON, OUT p_id BIGINT);       -- insert_asset(payload)
```

Rows are written via the stored procedure insert_asset, which accepts the full JSON string and returns the inserted row id as an OUT parameter. Reads use plain SELECT statements via the dictionary cursor.

Table: prices

Stores pricing results and market data series. The provider column is the discriminator that separates internally generated pricing output (provider = "INTERNAL") from externally fetched equity time series (provider = "eodhd"). This allows prices from different origins to coexist for the same code.

```
CREATE TABLE prices (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    code        VARCHAR(64)  NOT NULL,    -- ISIN, ticker, or internal ID
    provider    VARCHAR(32)  NOT NULL,    -- "INTERNAL" | "eodhd"
    json        JSON         NOT NULL     -- price payload
);
```

Key operations:

```
SELECT code, json FROM prices WHERE provider='INTERNAL';           --
select_prices()
SELECT code, json FROM prices WHERE provider='eodhd';              --
select_timeseries()
SELECT code, json FROM prices WHERE provider='INTERNAL'
  AND code=%s LIMIT 1;                                             --
select_price(code)
CALL insert_prices(p_json JSON);                                   --
insert_prices(payload)
```

Table: models

Stores one row per pricing model, holding the field schema that drives the Settings UI and the /fetch_models endpoint. required_fields and optional_fields are JSON arrays — each element is an object matching the structure in models/fields/<model>.json. This table is the live source of truth for field schemas; the JSON files on disk are the initial seed.

```
CREATE TABLE models (
    id               INT AUTO_INCREMENT PRIMARY KEY,
    name             VARCHAR(64)  NOT NULL UNIQUE,  -- model key, e.g. "hullwhite"
    required_fields  JSON,                          -- [{name, type, description,
...}]
    optional_fields  JSON                           -- [{name, type, default,
...}]
);
```

Key operations:

```
SELECT name, required_fields, optional_fields FROM models;   -- select_models()
CALL update_model(name, required_fields JSON,
                  optional_fields JSON);                      -- update_model()
```

Stored Procedures

Write operations are encapsulated in MySQL stored procedures to keep upsert / conflict logic server-side. Python calls them via cursor.callproc().

Procedure	Signature	Description
insert_asset	`insert_asset(IN p_json JSON, OUT p_id BIGINT)`	Upsert into assets by code; returns inserted id
insert_prices	`insert_prices(IN p_json JSON)`	Upsert into prices by code + provider
update_model	`update_model(IN p_name, IN p_req JSON, IN p_opt JSON)`	Update model field schemas by name

Row Decoding

All SELECT queries return rows via a dictionary cursor. The internal helper _decode_json_row() parses the json column (handling both bytes and str) and merges my_row_id and code as _id and _code into the returned dict, so callers always receive a flat Python dict regardless of query shape.

```
def _decode_json_row(row, json_field="json"):
    raw = row.get(json_field)
    obj = json.loads(raw.decode("utf-8") if isinstance(raw, bytes) else raw)
    if "my_row_id" in row: obj["_id"]   = row["my_row_id"]
    if "code"      in row: obj["_code"] = row["code"]
    return obj
```

API Integration

The FastAPI layer (api/main.py) calls db functions synchronously — there is no async ORM. Each endpoint that needs persistence calls the relevant db.* function directly. Read-heavy endpoints (fetch_assets, fetch_prices) accept the latency of a new connection per request; write endpoints (insert_asset, insert_prices, update_model) hold the connection only for the duration of the callproc() plus commit().

API Endpoint	db.py function	Direction
POST /assets	db.insert_asset()	Write — asset JSON from upload
GET /fetch_asset	db.select_asset(code)	Read — single asset by ISIN
GET /fetch_assets	db.select_assets()	Read — all assets
GET /fetch_models	db.select_models()	Read — all model schemas
POST /update_model	db.update_model()	Write — model field schema update
GET /fetch_prices	db.select_prices()	Read — all INTERNAL prices
GET /fetch_timeseries	db.select_timeseries()	Read — all eodhd price series
POST /insert_prices	db.insert_prices()	Write — pricing result payload

Hull-White Bond Pricer

```
model: "hullwhite"
```

Fixed / floating / callable investment-grade bonds

Description

The primary model for investment-grade corporate bonds, subordinated notes, senior unsecured debt, and covered bonds. Prices any combination of fixed coupon, floating (SOFR/Euribor + spread), fixed-to-float, and CMS-resettable coupons. Supports Bermudan and American call schedules, step-up/step-down coupon structures, and simultaneous price-to-worst-call and price-to-maturity valuation.

Pricing Methodology

Discounted cash flow (DCF) with a z-spread applied over the benchmark OIS curve. Each coupon and the redemption are discounted at:
DF*(t) = DF_curve(t) × exp(−z × t)

Floating coupons are projected using the simple forward rate extracted from the same discount curve:
fwd(t1,t2) = (DF(t1)/DF(t2) − 1) / yearFraction(t1,t2)

The call decision is handled by choosing the minimum of price-to-each-call date and price-to-maturity (price-to-worst). YTM and YTC are solved via bisection on the continuously compounded rate.

Key Formulas

```
NPV = Σ coupon(t) × DF*(t)  +  par × DF*(T)
DF*(t) = DF_curve(t) × exp(−z_spread × t)
fwd(t1,t2) = [DF(t1)/DF(t2) − 1] / yearFraction(t1, t2)
YTM: solve  Σ CF(t) × exp(−y × t) = NPV
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date (DD-MM-YYYY or YYYY-MM-DD)
issue_date	string	Issue date
maturity_date	string	Maturity date
fixed_coupon_rate	float	Annual coupon rate (decimal or %, auto-normalised)
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
accrual_day_count	string	30/360 | Actual365Fixed | ACT/ACT (ICMA) | Actual360
calendar	string	TARGET | UnitedStates
credit_spread_bp	float	Z-spread over benchmark curve in basis points

Optional Fields

Field	Type	Description
coupon_structure	string	fixed (default) | floating | fixed_to_float | cms_resettable
floating_spread_bp	float	Spread over reference rate for floating coupons (bp)
call_schedule	list	List of {date, price} call events for callable bonds
callable_type	string	bermudan | american
step_up_schedule	list	List of {date, rate} for step-up/step-down coupon changes
par	float	Face value (default 100)
settlement_days	int	Days to settlement (default 2)
currency	string	Settlement currency for curve selection (EUR, USD…)
redemption	float	Final redemption amount (default par)

Key Outputs

- npv / dirty_price, clean_price, accrued — in face-value units
- npv_to_maturity, npv_to_first_call, npv_to_worst_call
- ytm, ytc — continuously compounded yield solutions
- price_pct — all price metrics as % of par
- cashflows — per-period detail with DF, coupon, PV
- discount_curve_name — which curve was selected

Trinomial Tree Callable Bond Pricer

```
model: "trinomialtree"
```

Hull-White one-factor short-rate lattice for callable bonds

Description

Prices callable bonds by building a recombining trinomial lattice calibrated to the Hull-White one-factor short-rate model. At each node the issuer's call option is evaluated; the price is the risk-neutral expectation of future cash flows conditional on the optimal call decision. Supports the same coupon_structure values as the Hull-White DCF model.

Pricing Methodology

The short rate follows the Hull-White process:
$dr = [\theta(t) - a \times r]\, dt + \sigma\, dW$

A recombining trinomial lattice is built with spacing $\sqrt{(3\sigma^2 \Delta t)}$. At each callable date node, the issuer calls if the continuation value exceeds the call price. θ(t) is fit to the initial discount curve so that the lattice exactly reprices the benchmark curve.

Key Formulas

$$dr(t) = [\theta(t) - a \cdot r(t)]\, dt + \sigma\, dW(t)$$

$$\Delta r = \sigma\sqrt{(3\Delta t)}$$

$$\text{Call if: continuation\_value(node)} > \text{call\_price}$$

$$NPV = E^{Q}\left[\ \Sigma\ CF(t) \cdot \exp\left(-\int r\ ds\right)\ \right] \quad \text{(tree expectation)}$$

Required Fields

Field	Type	Description
instrument_id	string	Same required fields as hullwhite.py
maturity_date	string	Maturity date
fixed_coupon_rate	float	Annual coupon rate
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
credit_spread_bp	float	Z-spread over benchmark curve (bp)

Optional Fields

Field	Type	Description
hw_a	float	Hull-White mean reversion speed (default 0.03)
hw_sigma	float	Hull-White short-rate volatility (default 0.01)
tree_time_steps	int	Number of lattice time steps (default 120)
call_dates	list	List of callable dates (DD-MM-YYYY)
call_price	float	Call price as % of par (default par)
callable_type	string	bermudan | american

Key Outputs

- selected_npv — tree-computed price
- npv_to_maturity, npv_to_first_call, npv_to_worst_call
- ytm, ytc — continuously compounded yield solutions
- price_pct — all prices as % of par

Monte Carlo Hull-White Bond Pricer

```
model: "montecarlo"
```

Path simulation for callable and path-dependent bonds

Description

Prices callable bonds by simulating short-rate paths under the Hull-White model and discounting cash flows along each path. Preferred over the trinomial tree for long-dated callables or when path-dependent features make a lattice approach impractical. Supports the same coupon_structure values as the DCF and tree models.

Pricing Methodology

N paths of the Hull-White short rate are simulated using Euler discretisation. Along each path, cash flows are generated and discounted using the path's own numeraire. The call option is exercised on each path if the continuation value exceeds the call price. The NPV is the average discounted payoff across all paths.

Key Formulas

$$r(t+\Delta t) = r(t)\cdot\exp(-a\cdot\Delta t) + \alpha(t+\Delta t) + \sigma\cdot\sqrt{[(1-\exp(-2a\Delta t))/(2a)]}\cdot\varepsilon$$

$$\alpha(t) = f(0,t) + \sigma^2/(2a^2)\cdot(1-\exp(-at))^2$$

$$\text{Path DF}(0\to T) = \exp(-\Sigma\, r(t_i)\cdot\Delta t)$$

$$\text{NPV} = (1/N)\ \Sigma_{\text{paths}}\ [\ \Sigma\ \text{CF}(t)\cdot\text{PathDF}(t)\ ]$$

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
maturity_date	string	Maturity date
fixed_coupon_rate	float	Annual coupon rate
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
credit_spread_bp	float	Z-spread over benchmark curve (bp)

Optional Fields

Field	Type	Description
hw_a	float	Hull-White mean reversion (default 0.03)
hw_sigma	float	Hull-White volatility (default 0.01)
mc_time_steps	int	Time steps per path (default 360)
mc_num_paths	int	Number of paths (default 5000)
mc_seed	int	Random seed for reproducibility (default 42)
call_dates	list	List of callable dates (DD-MM-YYYY)

Key Outputs

- selected_npv, dirty_price, clean_price, accrued
- npv_to_maturity, npv_to_first_call, npv_to_worst_call
- ytm, ytc — yield solutions on the Monte Carlo price
- price_pct — all prices as % of par

Credit-Linked Note (CLN) Pricer

```
model: "cln"
```

Reduced-form hazard-rate model on CDS curve

Description

Prices a credit-linked note as a risky bond whose coupons and redemption are weighted by the survival probability of the reference entity. The hazard rate term structure is calibrated to CDS spreads from the curve file. A recovery-rate-weighted default leg adds the expected present value of recoveries. Yields are computed both on a promised basis (ignoring default) and on an expected basis (weighted by survival).

Pricing Methodology

CDS spreads are converted to piecewise-constant hazard rates:
$\lambda(t) = s(t) / (1 - R)$

Survival probability:
$S(t) = \exp(-\int_0^t \lambda(u)\, du)$

Default probability in interval $[t_{i-1}, t_i]$: $\Delta P = S(t_{i-1}) - S(t_i)$

Key Formulas

```
NPV = Σ coupon(t)·S(t)·DF(t)
    + par·S(T)·DF(T)
    + R·par·Σ ΔP(t)·DF_avg(t)
λ(t) = s(t) / (1 − R)          [hazard from CDS spread]
S(t) = exp(−Σ λᵢ·Δtᵢ)          [piecewise-constant λ]
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
issue_date	string	Issue date
maturity_date	string	Maturity date
fixed_coupon_rate	float	Annual coupon rate on the CLN
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
accrual_day_count	string	Day count convention
par	float	Face value (e.g. 100)
reference_entity	string	Name of reference entity (for CDS curve lookup)

Optional Fields

Field	Type	Description
recovery_rate	float	Default recovery rate (default 0.40)
credit_spread_bp	float	Issuer z-spread over risk-free rate (default 0)
reference_cds_curve_name	string	Explicit CDS curve override

Key Outputs

- selected_npv — total CLN NPV
- pv_coupons, pv_redemption, pv_recovery — component breakdown
- ytm_promised — yield ignoring default risk
- ytm_expected (= ytm) — yield weighted by survival / recovery
- hazard_segments — calibrated λ per tenor

- cds_curve_used — which CDS curve was selected

AT1 / CoCo Perpetual Capital Instrument Pricer

```
model: "at1"
```

Additional Tier 1 bank capital — fixed-to-reset perpetual

Description

Prices Additional Tier 1 (AT1) and Contingent Convertible (CoCo) perpetual bank capital instruments. Calculates two scenario prices: price-to-first-call (assuming the issuer calls at the first call date) and price-to-perpetuity (assuming extension, with coupons resetting to a forward par-swap rate plus the contractual reset spread). Extension risk, CET1 headroom, and loss absorption mode are reported.

Pricing Methodology

Phase 1 (issue → first_call_date): fixed coupon leg, standard DCF.

Phase 2 (first_call_date → perpetuity_horizon_years): the reset coupon is projected as the forward par-swap rate (from the benchmark curve) for the reset_reference_tenor starting at first_call_date, plus the contractual reset_spread.

Extension risk = (NPV_to_call − NPV_to_perpetuity) / par × 100

Price-to-worst = min(NPV_to_call, NPV_to_perpetuity).

Key Formulas

```
NPV_to_call     = Σ coupon·DF*(t)  +  par·DF*(T_call)
reset_coupon    = par_swap_rate(T_call, tenor) + reset_spread
NPV_to_perp     = NPV_to_call_leg + Σ_ext reset_coupon·DF*(t)
extension_risk = (NPV_to_call − NPV_to_perp) / par × 100  [%]
distance_to_trigger = cet1_current_pct − cet1_trigger_pct  [pp]
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
issue_date	string	Issue date
first_call_date	string	First issuer call date
fixed_coupon_rate	float	Fixed coupon rate until first call
coupon_frequency	string	Annual | Semiannual | Quarterly
accrual_day_count	string	Day count convention
calendar	string	TARGET | UnitedStates
credit_spread_bp	float	Z-spread in basis points
reset_spread	float	Contractual reset spread post-call (decimal or %)
reset_reference_tenor	string	Reset swap tenor (e.g. "5Y", "10Y")
cet1_trigger_pct	float	CET1 ratio trigger for loss absorption (%)
loss_absorption	string	write_down | equity_conversion

Optional Fields

Field	Type	Description
cet1_current_pct	float	Current CET1 ratio — for distance_to_trigger
conversion_price	float	Equity conversion price (equity_conversion type only)
perpetuity_horizon_years	float	Years to model post-call extension (default 50)
call_frequency_years	float	Subsequent call frequency after first call (default 1)

Key Outputs

- npv_to_first_call — price assuming call at first_call_date

- npv_to_perpetuity — price assuming extension with reset coupon
- selected_npv / npv_to_worst_call — min of the two
- reset_coupon_rate — projected reset coupon after first call
- reference_swap_rate — forward par-swap rate used
- extension_risk_pct — price difference call vs perpetuity
- distance_to_trigger — CET1 headroom above trigger (pp)

Discount Note Pricer

```
model: "discount_note"
```

Repos, commercial paper, T-bills, short-term instruments

Description

Money-market instrument pricer covering zero-coupon discount instruments (repos, Treasury bills, commercial paper) and add-on interest-bearing instruments (short-term FRNs, overnight loans). Computes discount rate, simple yield, and continuously compounded YTM, and reports a full accretion schedule from evaluation date to maturity.

Pricing Methodology

For discount instruments: the security is issued at PV and redeems at face_value at maturity. The discount rate is the bank-discount convention (return on face). Simple yield expresses the return on the purchase price.

For interest_bearing instruments: maturity_CF = face_value × (1 + coupon_rate × t).

YTM is continuously compounded: PV = maturity_CF × exp(−ytm × t).

Discounting uses the benchmark OIS curve with a z-spread overlay.

Key Formulas

```
discount_rate = (FV − PV) / FV × basis / t_days    [bank discount]
simple_yield  = (FV − PV) / PV × basis / t_days    [simple return]
YTM (c.c.):    PV = FV × exp(−ytm × t)
interest_bearing: maturity_CF = FV × (1 + coupon_rate × t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
issue_date	string	Issue date
maturity_date	string	Maturity date
face_value	float	Face / redemption value
instrument_type	string	discount | interest_bearing

Optional Fields

Field	Type	Description
coupon_rate	float	Add-on interest rate (interest_bearing type only)
day_count	string	Actual360 (default) | Actual365Fixed | ACT/ACT (ICMA)
credit_spread_bp	float	Z-spread in basis points (default 0)
settlement_days	int	Days to settlement (default 0)
calendar	string	TARGET | UnitedStates
currency	string	Settlement currency
repo_collateral	object	Sub-object: instrument_id, market_value, haircut_pct

Key Outputs

- npv / dirty_price — present value of maturity cash flow
- discount_rate — bank-discount convention rate
- simple_yield — add-on yield on purchase price
- ytm — continuously compounded yield to maturity
- accretion_schedule — day-by-day accretion from eval to maturity

PIK / PIK-Toggle Bond Pricer

```
model: "pik"
```

Payment-in-kind bonds — full PIK and PIK-toggle structures

Description

Prices payment-in-kind bonds where interest is paid by accreting into the outstanding principal rather than in cash. Covers full PIK bonds (all interest capitalises each period — equivalent to a compounding zero-coupon) and PIK-toggle bonds (issuer elects each period to pay cash or PIK, or a blend, with an optional PIK step-up rate). Common in leveraged finance, mezzanine debt, and private credit.

Pricing Methodology

Period-by-period walk: for each coupon period [d0, d1] with opening principal P:

PIK accreted = P × pik_rate × pik_election × yearFraction
Cash coupon = P × coupon_rate × (1 − pik_election) × yearFraction
P_new = P + PIK accreted

Final redemption = P_new after the last period.

Cash coupons are discounted with the benchmark curve + z-spread. The redemption (accreted principal) is similarly discounted.

Key Formulas

```
P(t+1) = P(t) × [1 + pik_rate × pik_election × accrual]
cash_CF(t) = P(t) × coupon_rate × (1 − pik_election) × accrual
NPV = Σ cash_CF(t) × DF*(t)  +  P_final × DF*(T)
DF*(t) = DF_curve(t) × exp(−z_spread × t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
issue_date	string	Issue / closing date
maturity_date	string	Maturity date
instrument_type	string	full_pik | pik_toggle
coupon_rate	float	Base coupon / PIK accrual rate
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
accrual_day_count	string	30/360 | Actual365Fixed | Actual360
calendar	string	TARGET | UnitedStates
credit_spread_bp	float	Z-spread in basis points

Optional Fields

Field	Type	Description
par	float	Initial principal (default 100)
pik_election	float	PIK fraction 0.0–1.0 (pik_toggle only; default 1.0)
pik_rate	float	Explicit PIK rate if different from coupon_rate
pik_rate_step_up	float	Step-up spread added to coupon_rate to get pik_rate

Key Outputs

- initial_principal — par at issuance
- final_principal — accreted redemption at maturity
- total_pik_accreted — final_principal − initial_principal

- npv / dirty_price, clean_price, accrued
- ytm — continuously compounded
- accretion_schedule — per-period opening/closing principal, PIK accreted, cash coupon

ABS / MBS Tranche Pricer

```
model: "abs"
```

Asset-backed securities and mortgage-backed securities

Description

Monthly pool cash-flow model for ABS and MBS tranches. Generates the projected pool schedule using a CPR (flat) or PSA (ramp) prepayment model combined with CDR-based defaults and a lagged recovery mechanism. Applies a sequential waterfall: credit support absorbs losses first, then senior tranches receive principal before this tranche. Supports generic ABS (auto, credit card, student loans), agency MBS (Fannie/Freddie/Ginnie — no credit risk), and non-agency/private-label MBS.

Pricing Methodology

For each month m with opening pool balance B:

```
CPR(m): flat or PSA ramp = min((seasoning+m)/30,1)×6%×speed/100
SMM = 1-(1-CPR)^(1/12)  |  MDR = 1-(1-CDR)^(1/12)
Prepayment = (B - sched_P) × SMM
Defaults = B × MDR  |  Losses = Defaults × loss_severity
Recovery arrives after recovery_lag_months
```

Tranche waterfall: credit_support absorbs losses; senior balance is paid first; this tranche receives remaining principal.

Key Formulas

```
SMM = 1 − (1 − CPR(m))^(1/12)
CPR_PSA(m) = min((seasoning+m)/30, 1) × 6% × psa_speed/100
MDR = 1 − (1 − CDR)^(1/12)
WAL = Σ [t × principal(t)] / Σ principal(t)
NPV = Σ tranche_CF(t) × DF_curve(t) × exp(−z×t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
instrument_type	string	abs | mbs_agency | mbs_non_agency
pool_balance	float	Current outstanding pool balance
pool_wac	float	Weighted average coupon of the pool
pool_wam	int	Weighted average remaining maturity (months)
tranche_balance	float	Outstanding notional of the tranche
tranche_coupon	float	Pass-through / certificate rate
credit_spread_bp	float	Z-spread in basis points

Optional Fields

Field	Type	Description
pool_seasoning	int	Months elapsed since origination (default 0)
cpr	float	Flat annual CPR (default 6%)
cdr	float	Annual CDR (default 0 for agency; 2% for others)
loss_severity	float	LGD fraction (default 0.40)
prepayment_model	string	cpr | psa (default psa for MBS, cpr for ABS)
psa_speed	float	PSA speed % (default 100)
credit_support_pct	float	Subordination as % of pool balance (default 0)
senior_notes_balance	float	Balance of senior tranches above this one (default 0)
recovery_lag_months	int	Months between default and recovery (default 6)

Key Outputs

- npv / dirty_price — present value of tranche cash flows
- wal — weighted average life in years
- total_principal_returned — total principal recovered
- cashflows — monthly detail: interest, principal, loss, DF, PV
- price_pct — NPV as % of tranche_balance

CLO Tranche Pricer

`model: "clo"` · **`instrument_type: "clo_tranche"`**

Collateralised loan obligation — floating-rate, pool simulation

Description

Quarterly period-by-period simulation of a CLO tranche through two structural phases. The floating coupon is projected using forward rates extracted from the same OIS discount curve used by the Hull-White pricer. Pool-level CDR defaults reduce the pool balance; an OC (overcollateralisation) test every period determines whether the tranche receives its full interest or whether interest is diverted to senior paydown.

Pricing Methodology

Phase 1 — Reinvestment (eval → reinvestment_end_date):

```
Pool principal is reinvested; only CDR defaults reduce pool_balance.
Investors receive only the floating interest leg.
```

Phase 2 — Amortisation (reinvestment_end → maturity):

```
Pool amortises linearly over pool_wal years.
Principal distributed sequentially: senior tranches first.
```

OC test: oc_ratio = pool_balance / total_rated_notes.
If oc_ratio < oc_threshold and not senior: tranche interest deferred.

Key Formulas

```
fwd(t1,t2) = [DF(t1)/DF(t2) − 1] / yearFraction(t1, t2)
tranche_interest(t) = T × (fwd + tranche_spread) × accrual
pool_loss(t) = P × [1−(1−CDR)^accrual] × (1−recovery_rate)
OC ratio = pool_balance / total_rated_notes
If OC < threshold (non-senior): tranche_interest = 0  [diverted]
```

Required Fields

Field	Type	Description
`instrument_id`	string	ISIN or internal identifier
`evaluation_date`	string	Pricing date
`instrument_type`	string	clo_tranche
`maturity_date`	string	Legal final maturity
`reinvestment_end_date`	string	End of reinvestment period
`pool_par_balance`	float	Current pool par balance
`pool_was`	float	Pool weighted average spread over reference rate
`pool_cdr`	float	Annual CDR of the pool
`tranche_balance`	float	Outstanding notional of the tranche
`tranche_spread`	float	Spread over reference rate
`oc_threshold`	float	OC ratio trigger (e.g. 1.25 = 125%)
`credit_support_pct`	float	Subordination below this tranche as % of pool
`credit_spread_bp`	float	Additional z-spread for discounting (bp)

Optional Fields

Field	Type	Description
`pool_recovery_rate`	float	Loan recovery rate (default 0.65)
`pool_cpr`	float	Voluntary loan repayment rate (default 0.15)
`pool_wal`	float	Pool WAL post-reinvestment in years (default 3.0)
`equity_pct`	float	Equity tranche as % of pool (default 10%)
`tranche_is_senior`	bool	True = most senior rated tranche (default False)
`management_fee_bp`	float	Senior management fee bp p.a. (default 25)

Key Outputs

- npv — present value of tranche cash flows
- wal — weighted average life
- oc_test_failures — number of periods where OC test failed
- cashflows — quarterly detail: fwd_rate, interest, principal, OC ratio, phase
- total_principal_returned

Leveraged Loan Pricer

`model: "clo"` · `instrument_type: "leveraged_loan"`

Single amortising floating-rate loan (TLB / TLA / unitranche)

Description

Prices a single leveraged loan — term loan B (TLB), term loan A (TLA), or unitranche — using the same forward-rate projection engine as the CLO tranche pricer. No pool or waterfall is involved: the model prices the loan directly as a floating-rate amortising instrument. Supports a SOFR/Euribor floor and the standard TLB 1% p.a. scheduled amortisation with a bullet repayment at maturity.

Pricing Methodology

For each payment period [d0, d1]:

```
fwd = (DF(d0)/DF(d1) − 1) / yearFraction(d0, d1)
floored_rate = max(fwd, floor_rate)
interest = L × (floored_rate + spread) × accrual
amort = loan_balance × annual_amort_pct / periods_per_year
(last period: bullet = full remaining balance)
```

For a floating-rate loan discounted off the same benchmark curve used to project coupons, the z-spread equals the discount margin (DM). Floor-active periods are flagged with * in the output.

Key Formulas

```
floored_rate(t) = max(fwd(t), floor_rate)
interest(t)     = L(t) × [floored_rate(t) + spread] × accrual
amort_per_period = loan_balance × annual_amort_pct / periods_per_year
DM = z_spread  [for floating-rate, projected and discounted off same curve]
WAL = Σ [t × principal(t)] / Σ principal(t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
instrument_type	string	leveraged_loan
maturity_date	string	Loan maturity
loan_balance	float	Outstanding principal
spread	float	Spread over SOFR/Euribor (decimal or %)
credit_spread_bp	float	DM / z-spread in basis points

Optional Fields

Field	Type	Description
floor_rate	float	SOFR/Euribor floor (default 0 = no floor)
annual_amortisation_pct	float	Scheduled amort % of original balance p.a. (default 1.0 = TLB)
coupon_frequency	string	Quarterly (default) | Monthly | Semiannual
settlement_days	int	Days to settlement (default 2)
calendar	string	TARGET | UnitedStates (default TARGET)
currency	string	Settlement currency

Key Outputs

- npv — present value of loan cash flows
- wal — weighted average life

- dm_bp — discount margin (= z-spread for floating-rate)
- cashflows — per-period: fwd_rate, floored_rate (*), balance, interest, principal

SPIRE Structured Note Pricer

```
model: "spire"
```

Capital-protected notes — collateral + swap decomposition

Description

Prices capital-protected and partial-protection structured notes constructed as a collateral bond plus a swap overlay. The collateral bond (typically a zero-coupon or low-coupon government or SSA bond) provides the capital guarantee; the swap adjustments price the structured coupon and performance features. Supports fixed, autocallable, and Bermudan-callable structures.

Pricing Methodology

Note PV = Collateral PV + Swap adjustments − Issuer spread PV

The collateral bond is priced using the Hull-White DCF engine against its own discount curve. Swap adjustments represent the difference between the structured coupon (paid to the investor) and the natural yield of the collateral.

For autocallable structures, Monte Carlo simulation (GBM) determines the probability of early redemption at each observation date.

Key Formulas

```
Note PV = Collateral PV + Swap adjustments − Issuer spread PV
Collateral PV = Σ collateral_CF(t) × DF_collateral(t)
For autocall:  PV = E^Q[CF on earliest autocall date | GBM paths]
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
coupon_structure	string	Must be "fixed"
note_notional	float	Notional of the note
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
calendar	string	TARGET | UnitedStates
collateral	object	Collateral bond definition (see below)
collateral.principal_amount	float	Collateral principal
collateral.coupon_rate	float	Collateral coupon
collateral.maturity_date	string	Collateral maturity

Optional Fields

Field	Type	Description
credit_spread_bp	float	Issuer spread on the note (default 0)
collateral_spread_bp	float	Spread on the collateral leg (default 0)
issuer_call	string	"Applicable" to enable call schedule
call_dates	list	List of call dates (DD-MM-YYYY)
callable_type	string	bermudan | autocallable
autocall_trigger	object	trigger_level_pct, mc_paths, mc_vol
valuation_mode	string	to_maturity | to_first_call | to_worst_call

Key Outputs

- selected_npv — structured note price
- npv_to_maturity, npv_to_first_call, npv_to_worst_call
- collateral_pv — standalone collateral bond value

- swap_adjustment — difference between structured and collateral leg
- cashflows — per-period detail

Index-Linked / CMS Channel Note Pricer

```
model: "index_linked"
```

Inflation-index-linked structured channel notes

Description

Prices structured notes where both coupons and redemption are linked to an inflation index ratio. The index ratio is projected forward from a known current level using a flat annual inflation assumption. Designed for structured retail products with a capital-protection element and an index-linked upside, such as inflation-linked channel notes issued by banks.

Pricing Methodology

The index ratio at any future date is projected as:

IndexRatio(d) = index_ratio_at_eval × (1 + annual_index_growth_rate)^t

Each structured coupon is multiplied by the index ratio at its payment date.

Key Formulas

```
IndexRatio(d) = IR_eval × (1 + g)^yearFraction(eval, d)
coupon(t) = coupon_multiplier × IR(t) × notional × accrual
Note PV = Collateral PV + indexed_swap_adjustment − issuer_spread_PV
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
coupon_structure	string	Must be "index_linked"
note_notional	float	Notional of the note
collateral	object	Collateral bond definition (same as SPIRE)
index_linked_assumption	object	Container for inflation assumptions
index_linked_assumption.index_ratio_at_eval	float	Current index ratio
index_linked_assumption.annual_index_growth_rate	float	Forward inflation rate
index_linked_assumption.coupon_multiplier	float	Real coupon multiplier

Optional Fields

Field	Type	Description
credit_spread_bp	float	Issuer spread (default 0)
collateral_spread_bp	float	Collateral leg spread (default 0)
issuer_call	string	"Applicable" to enable call schedule
call_dates	list	List of call dates (DD-MM-YYYY)

Key Outputs

- selected_npv — index-linked note price
- npv_to_maturity, npv_to_first_call, npv_to_worst_call
- index_ratio_at_maturity — projected final index ratio
- cashflows — per-period including index-adjusted coupons

Inflation-Linked Bond Pricer

```
model: "inflation_linked"
```

Government linkers — TIPS, OATi, BTPi, UK Gilts

Description

Plain-vanilla government-style inflation-linked bond pricer. Covers US TIPS, French OATi/OAT€i, Italian BTP-i, UK Gilts, and similar instruments where both coupons and redemption are scaled by the ratio of the current CPI to the base CPI at issuance. The redemption is optionally floored at par (standard for TIPS). Also computes breakeven inflation if a nominal yield is provided.

Pricing Methodology

Forward index ratios are projected from the current known ratio using a flat annual inflation assumption:

IndexRatio(d) = (current_cpi / base_cpi) × (1 + annual_inflation)^t

Each real coupon and the real redemption are multiplied by the index ratio at the payment date, then discounted with the benchmark curve + z-spread. Breakeven inflation is the rate that equates the inflation-adjusted NPV to the price implied by the nominal yield.

Key Formulas

```
IndexRatio(d) = (current_cpi/base_cpi) × (1+π)^yearFraction(eval,d)
coupon(t) = real_coupon × IndexRatio(t) × par × accrual
redemption = max(par, par × IndexRatio(T))  [if redemption_floor=true]
NPV = Σ coupon(t)·DF*(t)  +  redemption·DF*(T)
breakeven: solve π* s.t. NPV(π*) = par / (1+nominal_yield)^T
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
issue_date	string	Issue date
maturity_date	string	Maturity date
real_coupon_rate	float	Annual real coupon rate
base_cpi	float	CPI at issuance (index ratio denominator)
current_cpi	float	Current / latest CPI level
annual_inflation_rate	float	Forward flat inflation assumption
coupon_frequency	string	Annual | Semiannual | Quarterly | Monthly
accrual_day_count	string	ACT/ACT (ICMA) typical for linkers
calendar	string	TARGET | UnitedStates

Optional Fields

Field	Type	Description
par	float	Face value (default 100)
credit_spread_bp	float	Z-spread in basis points (default 0)
redemption_floor	bool	Floor accreted redemption at par (default True)
nominal_yield	float	If provided, breakeven inflation is computed
settlement_days	int	Days to settlement (default 2)

Key Outputs

- npv / dirty_price, clean_price, accrued
- index_ratio_at_maturity — projected final CPI ratio

- inflation_accreted_redemption — par × IndexRatio(T)
- breakeven_inflation — if nominal_yield was provided
- ytm — continuously compounded real yield
- cashflows — per-period real coupons × index ratio

Barrier Reverse Convertible Pricer

```
model: "barrier_convertible"
```

Worst-of, continuous barrier, Monte Carlo — SSPA product type 1230

Description

Prices barrier reverse convertible notes on one or multiple underlyings using GBM paths with optional Cholesky correlation for multi-asset baskets. The knock-in barrier is monitored continuously over the observation window. If never breached, the investor receives the full denomination at maturity; if breached, physical delivery of the worst performer (at the conversion ratio) is made instead.

Pricing Methodology

N correlated GBM paths are simulated for each underlying:

$dS_i/S_i = (r - q_i)\,dt + \sigma_i\,dW_i$

Cholesky decomposition of the correlation matrix produces correlated Brownian increments. For each path: barrier breached if any S_i(t) falls below barrier_i = initial_fixing_i × barrier_pct/100. The worst performer at maturity determines the payoff. Coupons are fixed amounts discounted along each path.

Key Formulas

```
dS_i/S_i = (r − q_i)·dt + σ_i·dW_i        [GBM per underlying]
Corr: dW = L·dZ  where L = Cholesky(ρ)
Payoff: denomination if barrier never hit
        else: min_i(conversion_ratio_i × S_i(T))
NPV = E^Q[Σ coupon_t·DF(t) + payoff_T·DF(T)]
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
maturity_date	string	Maturity date
denomination	float	Notional / face value
coupon_schedule	list	List of {date, amount} coupon payments
underlyings	list	List of underlying objects (see below)
underlyings[].initial_fixing	float	Spot at trade date
underlyings[].current_price	float	Current spot price
underlyings[].volatility	float	Annualised volatility
underlyings[].barrier_pct	float	Barrier as % of initial fixing
underlyings[].strike_pct	float	Strike as % of initial fixing

Optional Fields

Field	Type	Description
mc_num_paths	int	Number of Monte Carlo paths (default 5000)
mc_time_steps	int	Time steps per path (default 360)
mc_seed	int	Random seed (default 42)
correlation	list	N×N correlation matrix for multi-asset baskets
barrier_observation_period	object	start/end dates for barrier window override
issuer_spread_bp	float	Z-spread override

Key Outputs

- selected_npv — Monte Carlo price of the note

- barrier_breach_probability — % of paths where barrier was breached
- conditional_loss — average loss conditional on breach
- coupon_pv — PV of fixed coupon stream
- path_results — summary statistics across simulated paths

Barrier Discount Certificate Pricer

```
model: "barrier_discount"
```

European barrier at maturity, Monte Carlo — SSPA product type 1210

Description

Prices barrier discount certificates where the barrier is observed only at maturity (European observation), distinguishing it from the continuous barrier of the reverse convertible. If the final price is above the barrier, the investor receives the denomination; if below, physical delivery of the underlying at the conversion ratio. Supports a quanto adjustment for underlyings quoted in a different currency.

Pricing Methodology

N GBM paths are simulated for the single underlying. At maturity T only: if S(T) > barrier → denomination; else → conversion_ratio × S(T). The quanto adjustment corrects the drift when the stock is in a different currency from the settlement currency:

drift = r_settlement − q − quanto_adjustment
quanto_adjustment = ρ_SX × σ_S × σ_FX

Key Formulas

```
dS/S = (r − q − quanto_adj)·dt + σ·dZ      [single GBM path]
quanto_adj = ρ_SX × σ_S × σ_FX
Payoff(T): denomination if S(T) > barrier
           conversion_ratio × S(T) otherwise
NPV = DF(T) × E^Q[Payoff(T)]  +  Σ coupon_t × DF(t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
maturity_date	string	Maturity date
denomination	float	Notional / face value
underlyings	list	Single underlying object
underlyings[].initial_fixing	float	Spot at trade date
underlyings[].current_price	float	Current spot
underlyings[].volatility	float	Annualised volatility
underlyings[].barrier_pct	float	European barrier as % of initial fixing
underlyings[].strike_pct	float	Strike as % of initial fixing

Optional Fields

Field	Type	Description
quanto_adjustment	float	Quanto drift correction ρ·σ_S·σ_FX (default 0)
coupon_schedule	list	List of {date, amount} periodic coupons
mc_num_paths	int	Number of paths (default 5000)
mc_time_steps	int	Time steps per path (default 360)
mc_seed	int	Random seed (default 42)
issuer_spread_bp	float	Z-spread override

Key Outputs

- selected_npv — Monte Carlo price
- barrier_breach_probability — % paths where S(T) ≤ barrier

- expected_delivery_value — conditional delivery value if breached
- coupon_pv — PV of any periodic coupons

Simple Convertible / Reverse Convertible Pricer

```
model: "simple_convertible"
```

Worst-of, no barrier, Monte Carlo — standard and reverse payoffs

Description

Prices both reverse convertible and standard convertible notes on one or multiple underlyings without a barrier. For reverse convertibles, the embedded short put is the source of the elevated coupon. For standard convertibles, the embedded long call provides upside participation. Worst-of logic applies when multiple underlyings are present.

Pricing Methodology

N correlated GBM paths (Cholesky decomposition on correlation matrix). For each path at maturity:

Reverse: if worst_of < strike → deliver worst performer
`else → denomination (full cash repayment)`

Standard: if worst_of > strike → max(denomination, conversion × worst_of)
`else → denomination`

Key Formulas

```
Reverse payoff(T): denomination if worst_of(S_i(T)) ≥ strike_i
                   else: conv_ratio_worst × S_worst(T)
Standard payoff(T): max(denomination, conv_ratio_best × S_best(T))
                    where S_best is worst_of (long call on worst)
NPV = DF(T)·E^Q[Payoff(T)]  +  Σ coupon_t·DF(t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
maturity_date	string	Maturity date
denomination	float	Notional / face value
coupon_schedule	list	List of {date, amount} periodic coupons
underlyings	list	List of underlying objects
underlyings[].initial_fixing	float	Spot at trade date
underlyings[].current_price	float	Current spot
underlyings[].volatility	float	Annualised volatility
underlyings[].strike_pct	float	Strike as % of initial fixing

Optional Fields

Field	Type	Description
convertible_type	string	reverse (default) | standard
mc_num_paths	int	Number of paths (default 5000)
mc_time_steps	int	Time steps per path (default 360)
correlation	list	Correlation matrix for multi-asset baskets
issuer_spread_bp	float	Z-spread override

Key Outputs

- selected_npv — Monte Carlo price
- put_probability / call_probability — probability payoff is equity delivery vs denomination
- expected_equity_value — conditional value of worst-of at maturity

- coupon_pv — PV of periodic coupon stream

Autocallable Reverse Convertible Pricer

```
model: "autocallable_reverse_convertible"
```

Worst-of, continuous barrier, autocall, Monte Carlo — SSPA 1260

Description

Prices autocallable reverse convertible notes. At each autocall observation date, if all underlyings are at or above their respective autocall levels, the note redeems early at denomination. If never autocalled, the payoff at maturity is identical to the barrier reverse convertible: denomination if the barrier was never breached, or physical delivery of the worst performer if it was. Coupons accrue throughout the life of the trade regardless of early redemption.

Pricing Methodology

N correlated GBM paths. For each path and each autocall date (sorted ascending):

```
If ALL S_i(t_ac) ≥ autocall_level_i × initial_fixing_i:
  Redeem denomination at t_ac, discount to eval date, stop path.
```

If no autocall triggered:

```
Barrier continuously monitored → same worst-of payoff as barrier_convertible at T.
```

Coupons already paid up to early redemption are included.

Key Formulas

```
For each autocall date t_ac (ascending):
  If ∀i: S_i(t_ac) ≥ autocall_level_i → CF = denomination·DF(t_ac)
If no autocall:
  Barrier breached → worst_of(conv_ratio_i × S_i(T))·DF(T)
  Barrier clean    → denomination·DF(T)
NPV = E^Q[autocall_CF or maturity_CF]  +  Σ coupon_t·DF(t)
```

Required Fields

Field	Type	Description
instrument_id	string	ISIN or internal identifier
evaluation_date	string	Pricing date
maturity_date	string	Maturity date
denomination	float	Notional / face value
coupon_schedule	list	List of {date, amount} periodic coupons
autocall_schedule	list	List of {date, autocall_level_pct} observation events
underlyings	list	List of underlying objects
underlyings[].initial_fixing	float	Spot at trade date
underlyings[].current_price	float	Current spot
underlyings[].volatility	float	Annualised volatility
underlyings[].barrier_pct	float	Knock-in barrier as % of initial fixing
underlyings[].strike_pct	float	Final conversion strike as % of initial fixing

Optional Fields

Field	Type	Description
mc_num_paths	int	Number of paths (default 5000)
mc_time_steps	int	Time steps per path (default 360)
mc_seed	int	Random seed (default 42)
correlation	list	Correlation matrix for multi-asset baskets
barrier_observation_period	object	start/end dates for barrier window override
issuer_spread_bp	float	Z-spread override

Key Outputs

- selected_npv — Monte Carlo price
- autocall_probability — % paths redeemed early (any date)
- autocall_probability_by_date — per observation-date autocall probability
- barrier_breach_probability — % paths where barrier was breached (non-autocalled)
- expected_life — probability-weighted maturity in years
- coupon_pv — PV of all coupon cash flows